

咖啡台 LED 控制 · TCP 自定义协议 v1.0.1

项	值
适用对象	咖啡台智能终端
版本	v1.0.1 (2026-09-15)
用途	上位机/客户端作为 TCP 客户端主动连接终端，通过自定义 JSON 协议控制 10 路 LED，终端执行完毕后回一个应答报文。
说明	本协议用于对外控制；另保留标准 Modbus TCP（端口 502）作为调试/兼容接口（见文末附录）。

1. 传输与连接

项	值
协议	TCP（面向连接）
监听端口	9001
连接方向	客户端（客户端）主动 connect → 终端（服务端）应答；连接后由客户端发命令
并发连接	最多 4 个；第 5 个及之后的连接，终端在 accept 后立即主动关闭（客户端表现为 connect 成功后马上收到 EOF），请客户端实现断开重试
帧格式	每条命令一个 JSON 对象 + 一个换行符 <code>\n</code> (LF, 0x0A)；终端每条应答同样是一个 JSON 对象 + <code>\n</code>
单条帧长度上限	1024 字节（含结尾 <code>\n</code> ）
空闲超时	300 秒内未收到任何字节，终端主动关闭该连接
文本编码	UTF-8，不允许 BOM

1.1 帧长度与超长/非法处理

- 终端按 `\n` 切分消息，单条命令最长 1024 字节（含 `\n`）。
- 若超过上限、或单行 JSON 非法：终端丢弃该行，回一条 `INVALID_JSON`，连接保持不断开，继续处理后续命令。

1.2 分帧与粘包 / 半包

- 终端端按行缓冲：收到完整 `\n` 前不解析。

- 因此允许：一条命令被拆分到多个 TCP 段（半包）、或多条命令粘在同一 TCP 段（粘包）。终端均按 `\n` 逐行切分处理，客户端无需特殊处理，只需保证每条命令以 `\n` 结尾。

1.3 空闲超时

- 终端不做心跳检测：若某连接 300 秒内未收到任何字节，终端主动关闭并回收。
- 客户端异常掉线（无 FIN）的会话同样最迟 300 秒后被回收。
- 客户端可周期性发送 `{"cmd": "ping"}` 保活。

1.4 请求顺序（流水线）

- 允许在收到应答前连续发送多条命令；终端按请求到达顺序逐条处理并按序返回应答。
- 多路复用/需要精确匹配时，建议携带 `seq`。

2. 消息通用结构

2.1 请求（客户端 → 终端）

```
{ "seq": 123, "cmd": "led_set", "ch": 1, "on": true }
```

字段	类型	必填	说明
seq	int	否	客户端自定序号；应答原样回传，用于请求-应答匹配。 若携带必须为整数，范围 <code>-2147483648..2147483647</code> (int32)。多数客户端习惯用递增非负值。
cmd	string	是	命令名（见第 4 节）
其余	—	按命令	各命令参数

2.2 应答（终端 → 客户端，执行完毕后发送）

成功：

```
{ "seq": 123, "ok": true, "cmd": "led_set", "data": { "ch": 1, "on": true } }
```

失败：

```
{ "seq": 123, "ok": false, "cmd": "led_set", "err": "INVALID_CHANNEL", "msg": "ch 需为 1..10" }
```

字段	类型	说明
seq	int?	若请求带了 <code>seq</code> 则原样回传，否则应答不含 <code>seq</code>
ok	bool	<code>true</code> 成功 / <code>false</code> 失败
cmd	string	对应命令名（无法解析时可能为 <code>""</code> ）

data	object?	成功时的业务数据，见各命令
err	string?	失败时的错误码，见第 5 节
msg	string?	失败时的中文说明（UTF-8）

编码/类型补充：所有报文（含中文 msg）使用 UTF-8，不允许 BOM。布尔严格性见第 3 节。

3. LED 通道与类型约定

- 共 10 路：通道号 CH1 ~ CH10（取值 1..10）。
- 布尔严格性：请求中的 on 字段只接受 JSON 布尔 true / false；传 1、0、“true”、“false”等其他类型一律按 INVALID_VALUE 拒绝。应答中的 on / pong 等固定输出 JSON 布尔。
- on 为布尔 true 表示点亮、false 表示熄灭。（1 / 0 仅用于附录 Modbus 寄存器取值，两者类型规则互不影响。）
- 与 Modbus 寄存器对应关系见附录（Modbus 地址 0x0000..0x0009 = CH1..CH10）。

4. 命令集

4.1 ping — 连接测试 / 心跳

请求：{ "cmd": "ping" }

成功应答 data: { "pong": true }

4.2 led_set — 单路开/关

请求：{ "cmd": "led_set", "ch": 1, "on": true }

成功应答 data: { "ch": 1, "on": true }

错误：INVALID_CHANNEL（ch 不在 1..10）、INVALID_VALUE（on 非布尔 true / false）

4.3 led_set_all — 全开 / 全关

请求：{ "cmd": "led_set_all", "on": true }

成功应答 data: { "count": 10, "on": true }

说明：count 为实际生效路数；on 为本次统一设置的状态（布尔）。

错误：INVALID_VALUE（on 非布尔）

4.4 led_set_batch — 多条一起设置（一次应答，原子）

请求：{ "cmd": "led_set_batch", "list": [{"ch":1,"on":true}, {"ch":3,"on":false}] }

成功应答 data: { "applied": 2 }

说明：

- 原子执行：先对整批做完整校验，全部合法才统一执行；任一条非法则返回错误、一条都不执行。因此 ok:true 时批内每条都已生效。

- `applied`：本次成功设置的条数。由于原子语义，成功时 `applied` 恒等于 `list` 长度，保留该字段仅作对账用途。
- `list` 长度 1..10 条；缺失、非数组、空数组或超过 10 条均返回 `INVALID_LIST`。
- 同一 `ch` 在批内重复：按“后者生效”处理（按数组顺序逐条应用），不算错误。
- 元素格式：每项必须同时含合法 `ch` 与布尔 `on`，否则按具体原因返回 `INVALID_CHANNEL` / `INVALID_VALUE` / `INVALID_LIST`。

错误： `INVALID_CHANNEL`、`INVALID_VALUE`、`INVALID_LIST`

4.5 `led_status` — 读当前全部状态

请求： `{ "cmd": "led_status" }`

成功应答 `data`：

```
{ "channels": [ {"ch":1,"on":true}, {"ch":2,"on":false}, ... ] }
```

说明： `channels` 固定为 10 项，顺序即 CH1..CH10； `on` 恒为布尔。

4.6 状态与掉电行为

- LED 通道状态仅保存在运行内存中，终端断电/重启后所有通道恢复为灭（不支持断电记忆）。
- 适用场景：终端支持灵活的动态效果（流水、呼吸等）高频控制。
- 如需“开机记忆”，请联系厂商定制（采用带写入节流的保存策略，不影响本协议字段）。

5. 错误码

err	说明
<code>INVALID_JSON</code>	JSON 无法解析 / 空串 / 命令超长
<code>INVALID_CMD</code>	<code>cmd</code> 缺失或未知
<code>INVALID_CHANNEL</code>	<code>ch</code> 不在 1..10
<code>INVALID_VALUE</code>	<code>on</code> 不是布尔 <code>true</code> / <code>false</code> ，或数值非法
<code>INVALID_LIST</code>	<code>list</code> 缺失、非数组、空数组或超过 10 条 / 数组元素格式非法
<code>INTERNAL</code>	终端内部异常

6. 交互示例

`->` 为客户端发送，`<-` 为终端应答。客户端 `connect 192.168.1.100:9001`。

点亮 CH3（带 seq，回传匹配）

```
-> {"seq":1,"cmd":"led_set","ch":3,"on":true}
<- {"seq":1,"ok":true,"cmd":"led_set","data":{"ch":3,"on":true}}
```

读全部状态（本请求未携带 seq，故应答中亦无 seq 字段——seq 仅原样回传，请求不带则应答不出现）

```
-> {"cmd":"led_status"}
<- {"ok":true,"cmd":"led_status","data":{"channels":[{"ch":1,"on":true},{"ch":2,"on":false},
{"ch":3,"on":true},{"ch":4,"on":false},{"ch":5,"on":false},{"ch":6,"on":false},{"ch":7,"on":false},
{"ch":8,"on":false},{"ch":9,"on":false},{"ch":10,"on":false}]}}
```

非法通道

```
-> {"seq":2,"cmd":"led_set","ch":11,"on":true}
<- {"seq":2,"ok":false,"cmd":"led_set","err":"INVALID_CHANNEL","msg":"ch 需为 1..10"}
```

粘包示例（一次发两条，按序回两条应答）

```
-> {"cmd":"led_set","ch":1,"on":true}{"cmd":"led_set","ch":2,"on":true}
<- {"ok":true,"cmd":"led_set","data":{"ch":1,"on":true}}
<- {"ok":true,"cmd":"led_set","data":{"ch":2,"on":true}}
```

7. 安全说明

- 本协议 v1.0.1 无鉴权/令牌：连上即可控制全部 LED，且终端入网后 9001 常驻监听。请部署于可信局域网 / VLAN，或用交换机/防火墙做网段隔离。
- 若终端将部署到客户现场网络（非受控局域网），建议在量产前启用 v1.1 访问令牌，不要仅依赖 VLAN 隔离。
- 在令牌方案落地前，至少应向集成商明示“9001 与 502 端口无鉴权”这一事实。

附录 A · 与 Modbus TCP 从站（端口 502，调试用）

终端同时在 端口 502 提供标准 Modbus TCP 从站（用于调试/兼容）：

Modbus 项	值
功能码	0x03 读保持寄存器 / 0x06 写单个 / 0x10 写多个
寄存器地址	0x0000..0x0009 = CH1..CH10
寄存器值	0 =灭, 1 =亮
工具	mbpoll / Modbus Poll / pymodbus

并发写注意：Modbus 从站与自定义 TCP 协议（9001）可同时监听、互不干扰（均只占用网络端口）。但两套接口操作的是同一组通道状态，并发写入时以后到达者为准（last-write-wins），无锁与原子组合语义。调试时请先停止上位机对 9001 的下发，再用 Modbus 工具手动读写，避免互相覆盖造成误判。Modbus 寄存器值 0 / 1 对应灭/亮，与 9001 的布尔字段互不影响（各自遵循各自协议的类型规则）。